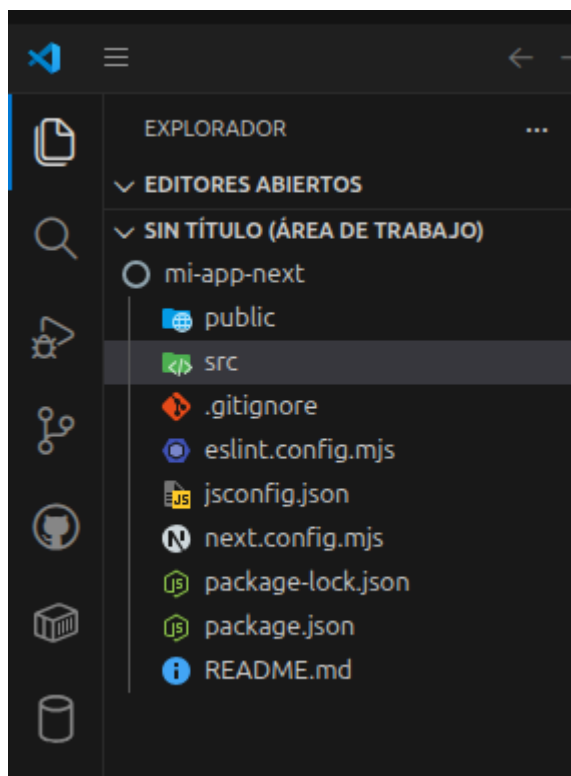
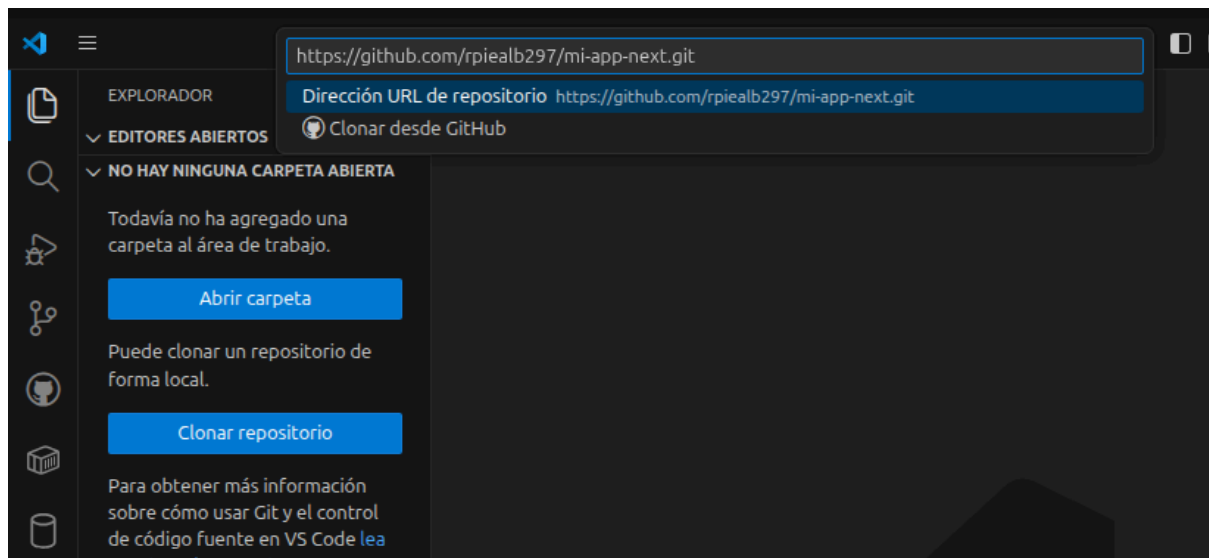


Paso 1 - Clona y entra al proyecto:



Paso 2 - Instala las dependencias de Swagger:

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS  bash + v
usuario@usuario:~/Descargas/mi-app-next$ npm install swagger-ui-react swagger-
jsdoc
```

Paso 3 - Crea/modifica los archivos

Archivo 1 de 6 - Configuración de Swagger

Crea la carpeta `src/lib/` y dentro el archivo `swagger.js`:

Ruta: `src/lib/`[swagger.js](#)

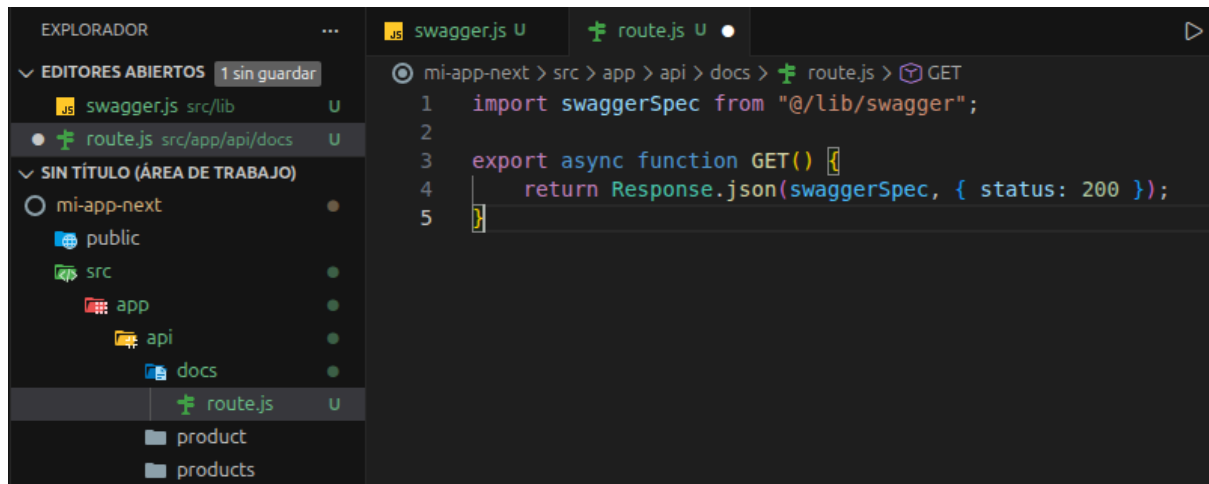
```
EXPLORADOR  ...  swagger.js U
EDITORES ... 1 sin guardar
  swagger.js sr... U
SIN TÍTULO (ÁREA DE TRA...
mi-app-next
  public
  src
    app
      api
      favicon.ico
      globals.css
      layout.js
      page.js
      page.modu...
    lib
      swag... U
.gitignore
eslint.config.mjs
jsconfig.json
next.config.mjs
package-loc... M
package.json M
README.md

mi-app-next > src > lib > swagger.js > default
1  import swaggerJsdoc from "swagger-jsdoc";
2
3  const options = {
4    definition: {
5      openapi: "3.0.0",
6      info: {
7        title: "Mi API Next.js",
8        version: "1.0.0",
9        description: "Documentación de la API creada con Next.js y
10         Swagger",
11      },
12      servers: [
13        {
14          url: "http://localhost:3000",
15          description: "Servidor de desarrollo",
16        },
17      ],
18    },
19    apis: ["./src/app/api/**/*.js"],
20  };
21
22  const swaggerSpec = swaggerJsdoc(options);
23  export default swaggerSpec;
```

Archivo 2 de 6 - Endpoint que sirve el JSON de Swagger

Crea la carpeta `src/app/api/docs/` y dentro el archivo `route.js`:

Ruta: `src/app/api/docs/`[route.js](#)



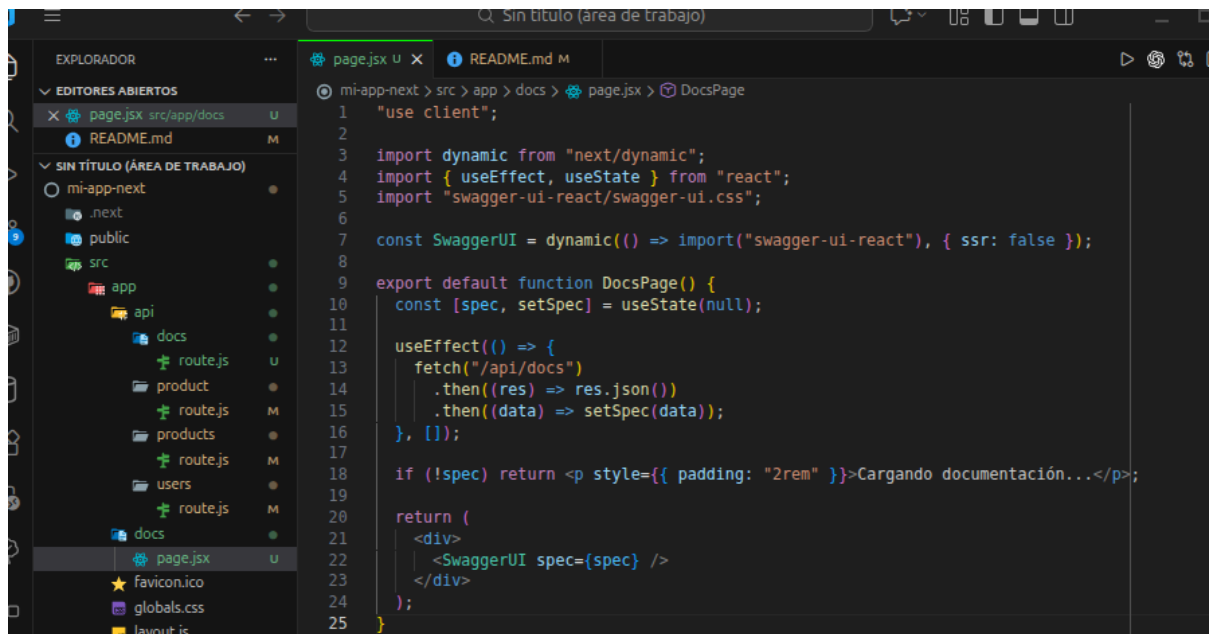
The screenshot shows the VS Code interface. On the left, the Explorer sidebar displays the project structure: `mi-app-next` > `public` > `src` > `app` > `api` > `docs`. The file `route.js` is selected. The main editor shows the content of `route.js`:

```
1 import swaggerSpec from "@/lib/swagger";
2
3 export async function GET() {
4   return Response.json(swaggerSpec, { status: 200 });
5 }
```

Archivo 3 de 6 - Página visual de Swagger UI

Crea la carpeta `src/app/docs/` y dentro el archivo `page.jsx`:

Ruta: `src/app/docs/`[page.jsx](#)



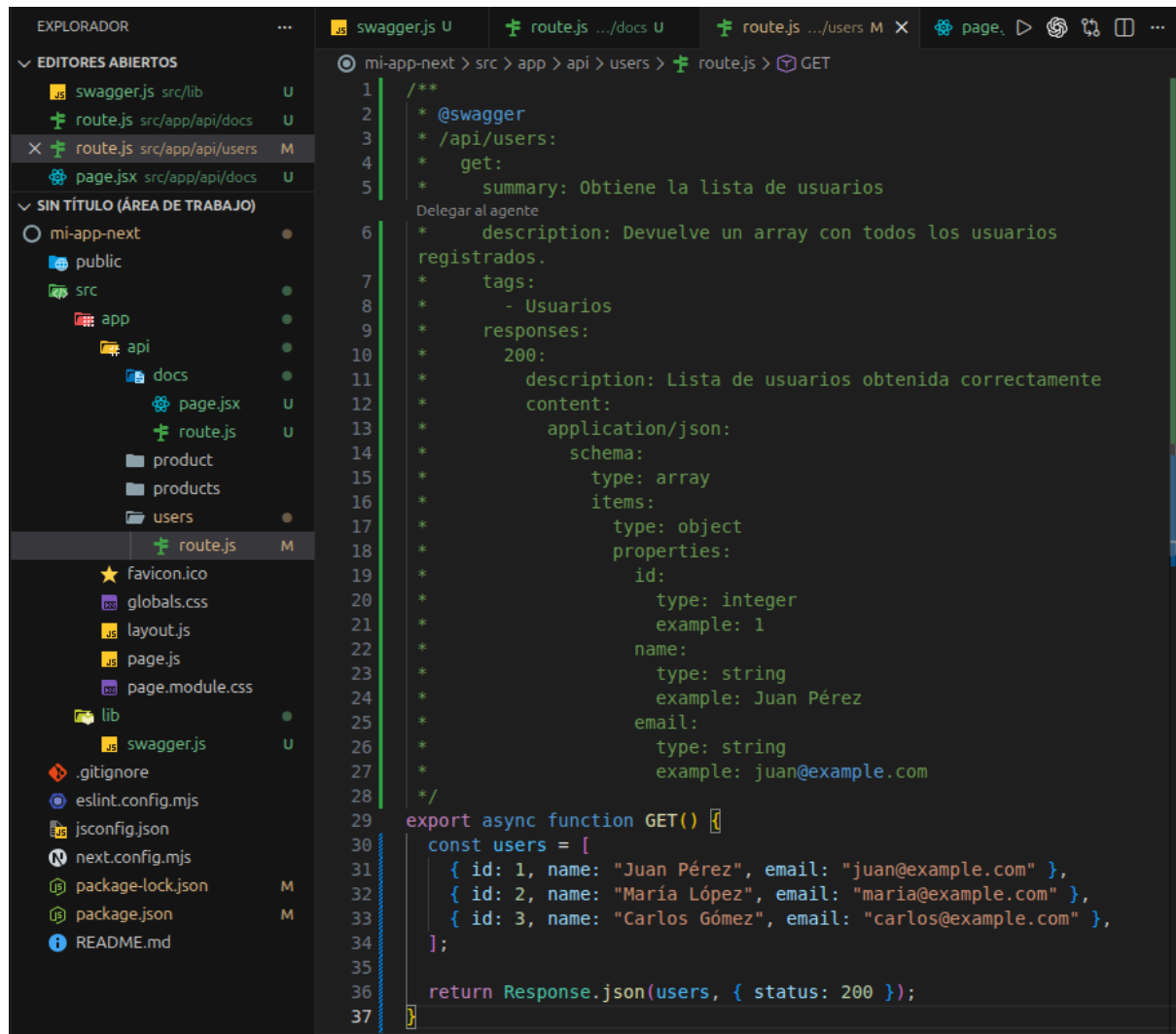
The screenshot shows the VS Code interface. On the left, the Explorer sidebar displays the project structure: `mi-app-next` > `public` > `src` > `app` > `docs`. The file `page.jsx` is selected. The main editor shows the content of `page.jsx`:

```
1 "use client";
2
3 import dynamic from "next/dynamic";
4 import { useEffect, useState } from "react";
5 import "swagger-ui-react/swagger-ui.css";
6
7 const SwaggerUI = dynamic(() => import("swagger-ui-react"), { ssr: false });
8
9 export default function DocsPage() {
10   const [spec, setSpec] = useState(null);
11
12   useEffect(() => {
13     fetch("/api/docs")
14       .then((res) => res.json())
15       .then((data) => setSpec(data));
16   }, []);
17
18   if (!spec) return <p style={{ padding: "2rem" }}>Cargando documentación...</p>;
19
20   return (
21     <div>
22       <SwaggerUI spec={spec} />
23     </div>
24   );
25 }
```

Archivo 4 de 6 — Endpoint users (modificar el existente)

Este archivo ya existe, solo tienes que sustituir todo su contenido:

Ruta: `src/app/api/users/route.js`

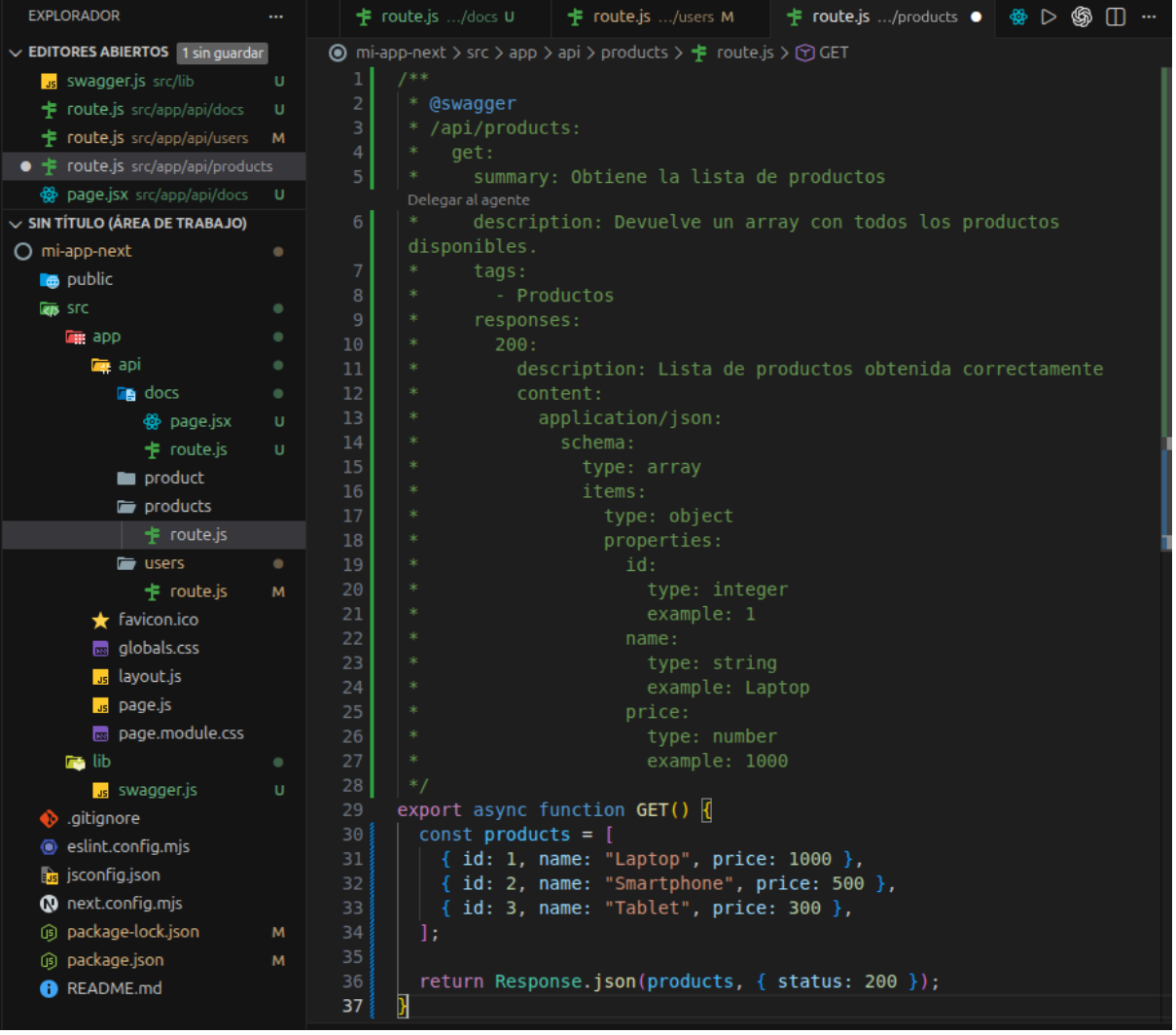


The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows the project structure, with the file `route.js` selected in the `src/app/api/users` directory. The code editor shows the content of `route.js`, which is a Next.js API route for the `/api/users` endpoint. The code includes a Swagger comment and a GET handler that returns a list of users.

```
1 /**
2  * @swagger
3  * /api/users:
4  *   get:
5  *     summary: Obtiene la lista de usuarios
6  *     description: Devuelve un array con todos los usuarios
7  *     tags:
8  *       - Usuarios
9  *     responses:
10 *       200:
11 *         description: Lista de usuarios obtenida correctamente
12 *         content:
13 *           application/json:
14 *             schema:
15 *               type: array
16 *               items:
17 *                 type: object
18 *                 properties:
19 *                   id:
20 *                     type: integer
21 *                     example: 1
22 *                   name:
23 *                     type: string
24 *                     example: Juan Pérez
25 *                   email:
26 *                     type: string
27 *                     example: juan@example.com
28 */
29 export async function GET() {
30   const users = [
31     { id: 1, name: "Juan Pérez", email: "juan@example.com" },
32     { id: 2, name: "María López", email: "maria@example.com" },
33     { id: 3, name: "Carlos Gómez", email: "carlos@example.com" },
34   ];
35
36   return Response.json(users, { status: 200 });
37 }
```

Archivo 5 de 6 — Endpoint products (modificar el existente)

Ruta: src/app/api/products/route.js

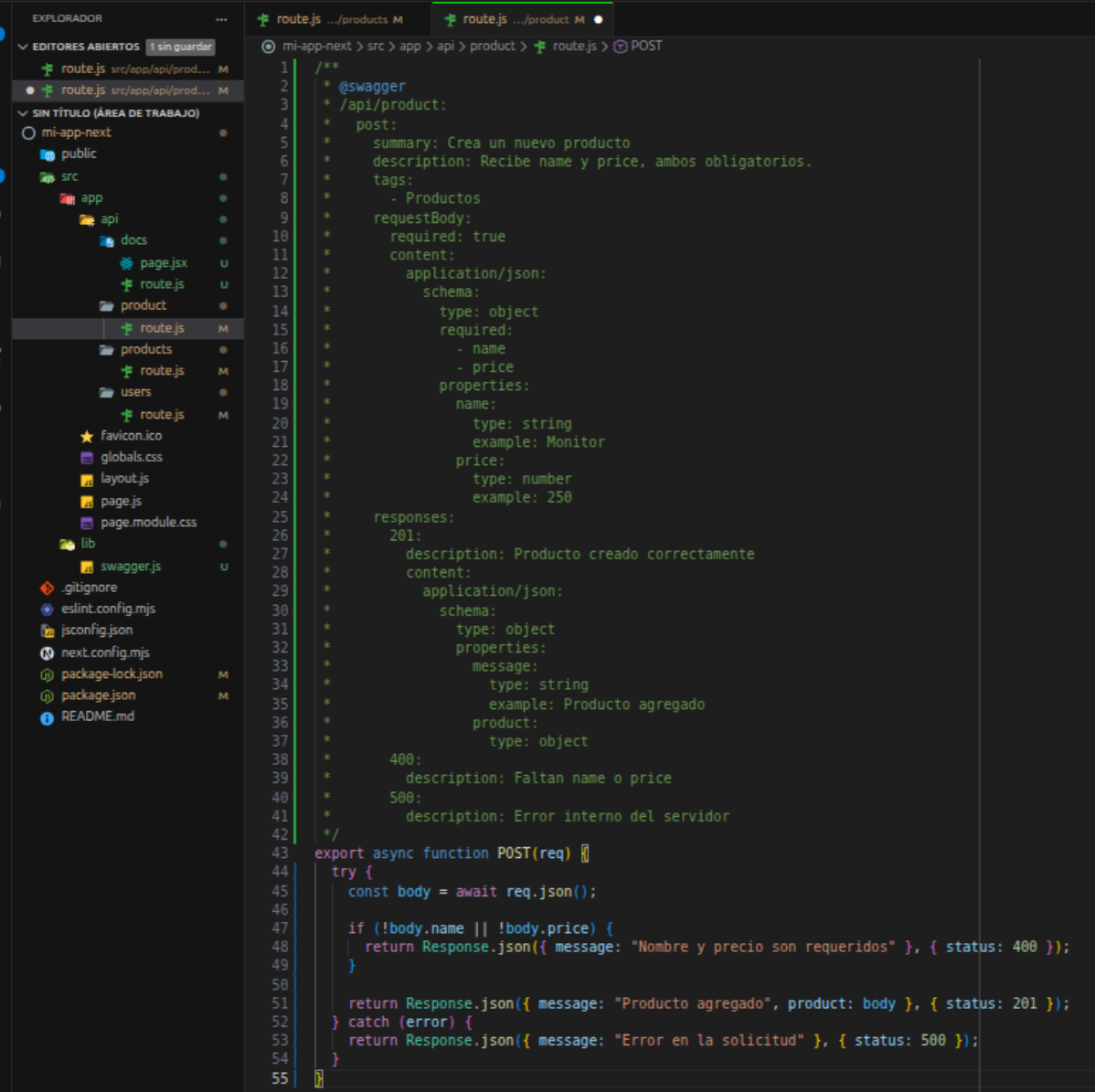


The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows the project structure, with the file `route.js` in the `src/app/api/products` directory selected. The code editor shows the content of `route.js`, which is a Next.js API route for the `/api/products` endpoint. The code includes a Swagger comment and an async function `GET()` that returns a JSON response with a list of products.

```
1  /**
2  * @swagger
3  * /api/products:
4  *   get:
5  *     summary: Obtiene la lista de productos
6  *     description: Devuelve un array con todos los productos disponibles.
7  *     tags:
8  *       - Productos
9  *     responses:
10 *       200:
11 *         description: Lista de productos obtenida correctamente
12 *         content:
13 *           application/json:
14 *             schema:
15 *               type: array
16 *               items:
17 *                 type: object
18 *                 properties:
19 *                   id:
20 *                     type: integer
21 *                     example: 1
22 *                   name:
23 *                     type: string
24 *                     example: Laptop
25 *                   price:
26 *                     type: number
27 *                     example: 1000
28 */
29 export async function GET() {
30   const products = [
31     { id: 1, name: "Laptop", price: 1000 },
32     { id: 2, name: "Smartphone", price: 500 },
33     { id: 3, name: "Tablet", price: 300 },
34   ];
35
36   return Response.json(products, { status: 200 });
37 }
```

Archivo 6 de 6 - Endpoint product POST (modificar el existente)

Ruta: src/app/api/product/route.js



```
1  /**
2  * @swagger
3  * /api/product:
4  *   post:
5  *     summary: Crea un nuevo producto
6  *     description: Recibe name y price, ambos obligatorios.
7  *     tags:
8  *       - Productos
9  *     requestBody:
10 *       required: true
11 *       content:
12 *         application/json:
13 *           schema:
14 *             type: object
15 *             required:
16 *               - name
17 *               - price
18 *             properties:
19 *               name:
20 *                 type: string
21 *                 example: Monitor
22 *               price:
23 *                 type: number
24 *                 example: 250
25 *     responses:
26 *       201:
27 *         description: Producto creado correctamente
28 *         content:
29 *           application/json:
30 *             schema:
31 *               type: object
32 *               properties:
33 *                 message:
34 *                   type: string
35 *                   example: Producto agregado
36 *                 product:
37 *                   type: object
38 *       400:
39 *         description: Faltan name o price
40 *       500:
41 *         description: Error interno del servidor
42 */
43 export async function POST(req) {
44   try {
45     const body = await req.json();
46
47     if (!body.name || !body.price) {
48       return Response.json({ message: "Nombre y precio son requeridos" }, { status: 400 });
49     }
50
51     return Response.json({ message: "Producto agregado", product: body }, { status: 201 });
52   } catch (error) {
53     return Response.json({ message: "Error en la solicitud" }, { status: 500 });
54   }
55 }
```

npm run dev

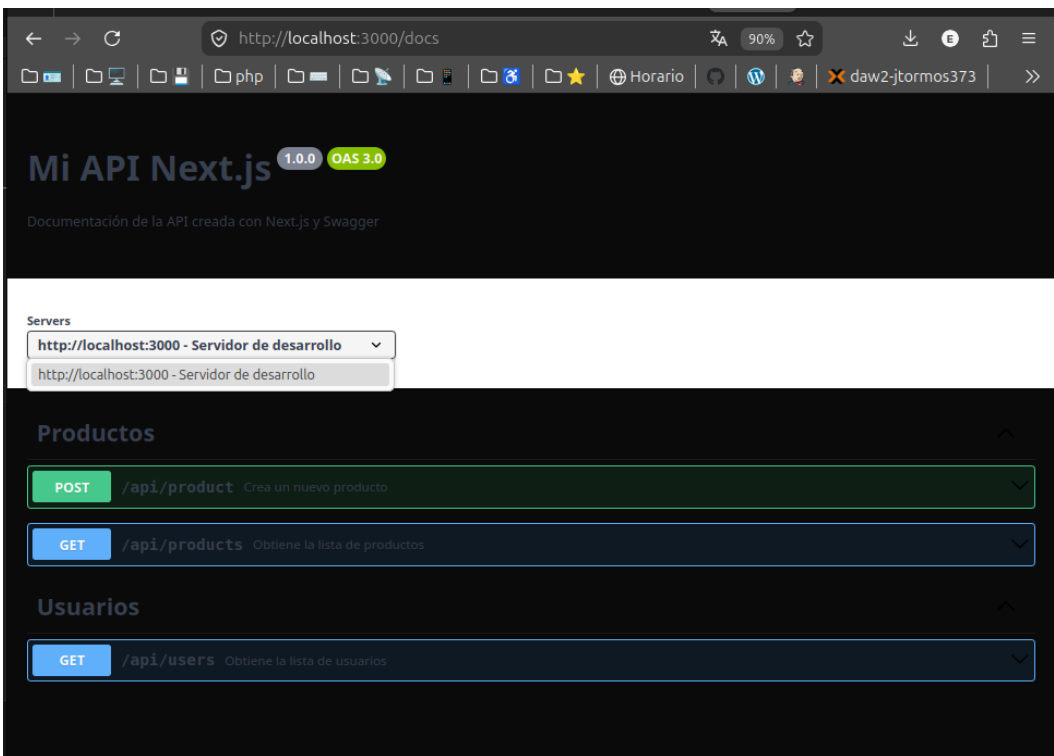
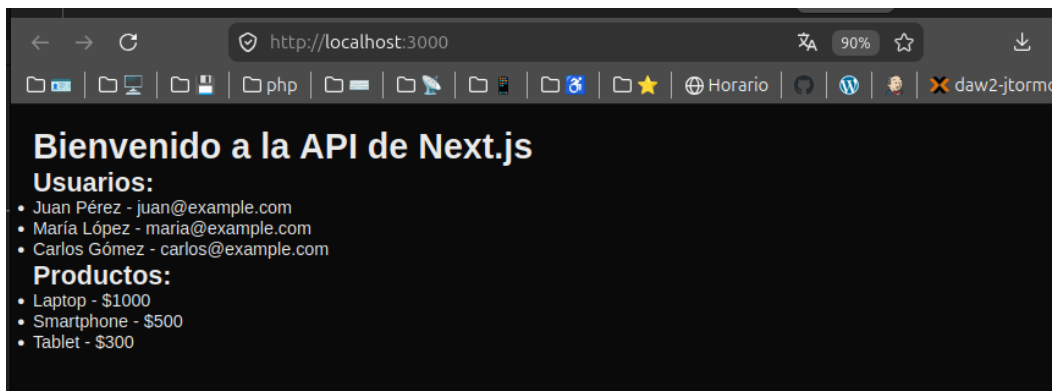
```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS  bash + v ...  bash

Severity: moderate
PostCSS has XSS via Unescaped </style> in its CSS Stringify Output - h
https://github.com/advisories/GHSA-qx2v-qp2m-jg93
fix available via `npm audit fix --force`
Will install next@9.3.3, which is a breaking change
node_modules/postcss
  next 9.3.4-canary.0 - 16.3.0-canary.5
  Depends on vulnerable versions of postcss
  node_modules/next

2 moderate severity vulnerabilities

To address all issues (including breaking changes), run:
  npm audit fix --force
usuario@usuario:~/Descargas/mi-app-next$ npm run dev
```

localhost + /docs



post + gets

post

The screenshot shows a web browser window with the address bar displaying `http://localhost:3000/docs`. The browser's address bar includes navigation icons, a search icon, a zoom level of 67%, and a star icon. Below the address bar is a bookmark bar with various icons and links, including 'php', 'Horario', and 'daw2-jtormos373'. The main content area of the browser displays a REST client interface. At the top, there is a text input field with the placeholder text 'Recibe name y price, ambos obligatorios.' Below this is a 'Parameters' section with a 'Cancel' button. The 'Request body' section is marked as 'required' and has a dropdown menu set to 'application/json'. The 'Edit Value' section shows a JSON object:

```
{  "name": "Monitor",  "price": 250}
```

. Below the request body is a blue 'Execute' button and a 'Clear' button. The 'Responses' section is at the bottom. It includes a 'Curl' section with the following command:

```
curl -X 'POST' \  'http://localhost:3000/api/product' \  -H 'accept: application/json' \  -H 'Content-Type: application/json' \  -d '{  "name": "Monitor",  "price": 250  }'
```

. Below the curl command is the 'Request URL' section with the value `http://localhost:3000/api/product`. The 'Server response' section shows a '201' status code and a 'Response body' containing a JSON object:

```
{  "message": "Producto agregado",  "product": {    "name": "Monitor",    "price": 250  }}
```

. A 'Download' button is visible at the bottom right of the response body.

users

GET

/api/users

Obtiene la lista de usuarios

Devuelve un array con todos los usuarios registrados.

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:3000/api/users' \
  -H 'accept: application/json'
```

Request URL

http://localhost:3000/api/users

Server response

Code

Details

200

Response body

```
[
  {
    "id": 1,
    "name": "Juan Pérez",
    "email": "juan@example.com"
  },
  {
    "id": 2,
    "name": "María López",
    "email": "maria@example.com"
  },
  {
    "id": 3,
    "name": "Carlos Gómez",
    "email": "carlos@example.com"
  }
]
```

Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Thu, 07 May 2026 10:37:51 GMT
keep-alive: timeout=5
transfer-encoding: chunked
vary: rsc,next-router-state-tree,next-router-prefetch,next-router-segment-prefetch
```

Responses

Code	Description	Links
200	Lista de usuarios obtenida correctamente	No links

Media type

application/json

Continue Accept Header

products

← → ↺

http://localhost:3000/docs

🔍 67% ☆

⬇️ ⓘ ⬆️ ☰

📁 🖨️ 📄 📁 php 📁 📧 📁 📱 📁 🌐 📁 ★ 🌐 Horario 🌐 🗑️ 🌐 WordPress 🌐 🤖 daw2-jtormos373 >>

GET /api/products Obtene la lista de productos

Devuelve un array con todos los productos disponibles.

Parameters Cancel

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:3000/api/products' \
-H 'accept: application/json'
```

Request URL

```
http://localhost:3000/api/products
```

Server response

Code Details

200

Response body

```
{
  {
    "id": 1,
    "name": "Laptop",
    "price": 1000
  },
  {
    "id": 2,
    "name": "Smartphone",
    "price": 500
  },
  {
    "id": 3,
    "name": "Tablet",
    "price": 300
  }
}
```

Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Thu, 07 May 2026 10:36:55 GMT
keep-alive: timeout=5
transfer-encoding: chunked
vary: rsc,next-router-state-tree,next-router-prefetch,next-router-segment-prefetch
```

Responses

code Description Links

